MALUBA MULEBI: 2025025124
COMPUTER VISION CSC6302 PRACTICAL 1

TASK 1: COLOUR SPACES AND A COLOUR PICKER

The cake image was loaded using OpenCV and converted from BGR to RGB, HSV and LAB. HSV represents hue, saturation and brightness, while LAB represents lightness and the green–red and blue–yellow color ranges [1], [2].

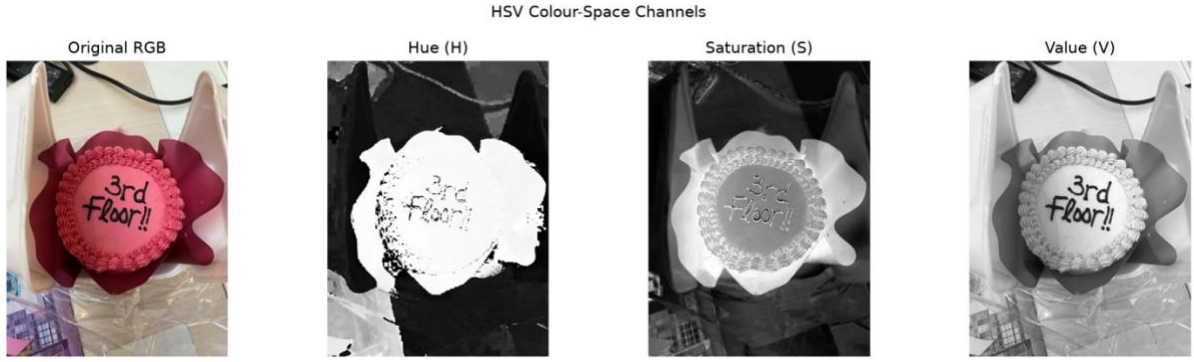

Figure 1: Original cake image and its HSV channels.

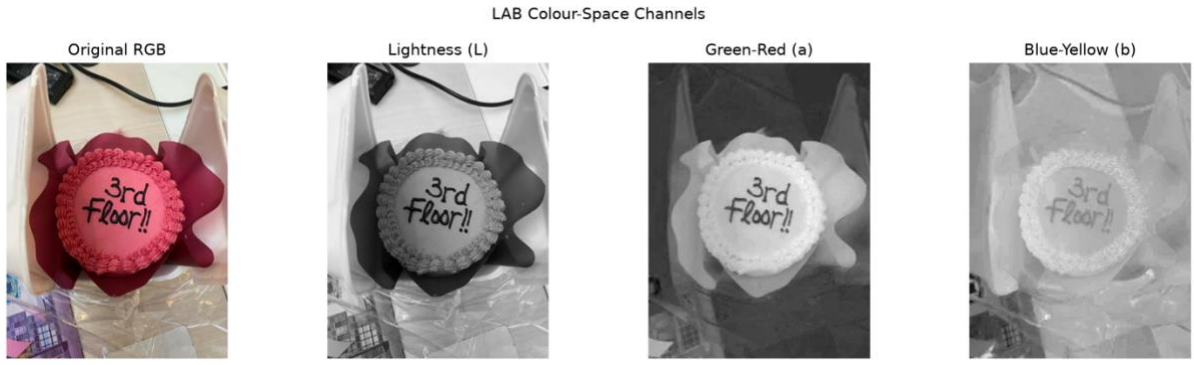

Figure 2: Original cake image and its LAB channels.

Pink and magenta regions were isolated using an HSV hue range of 155–179, with minimum saturation and value thresholds of 55. A binary mask was created using cv2.inRange and applied using cv2.bitwise_and.

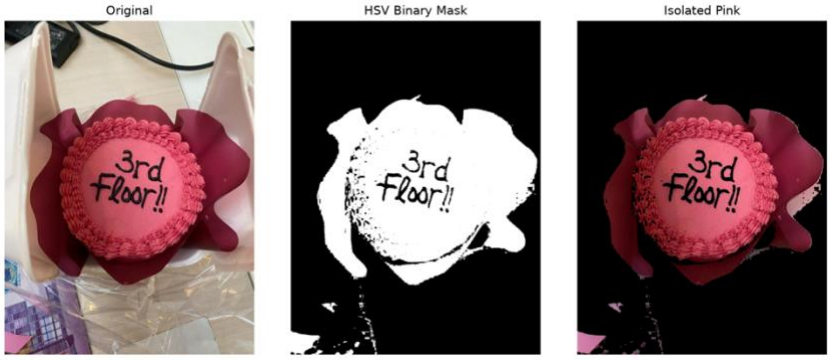

Figure 3: Original image, HSV binary mask and isolated pink regions.

HSV simplified color selection because hue can be thresholded separately from brightness. The mask captured most of the pink cake but also included parts of the burgundy wrapper because the two

regions had overlapping hue values. This shows that color thresholding cannot reliably distinguish different objects with similar colors.

TASK 2: NOISE AND SMOOTHING – A CONTROLLED EXPERIMENT

The grayscale cup image was corrupted with Gaussian noise (standard deviation 15) and salt-and-pepper noise affecting 5% of its pixels. Box, Gaussian and median filters were tested using 3×3, 5×5 and 7×7 kernels. A fixed random seed of 2025025124 ensured reproducible results.

PSNR was calculated as:

$$PSNR = 10 \log_{10}(255^2/MSE)$$

where MSE measures the difference between the original and restored images. A higher PSNR indicates better restoration.

Table 1: PSNR results for Gaussian and salt-and-pepper noise

Noise type	Filter	Kernel	PSNR (dB)
Gaussian	Box	3×3	30.54
Gaussian	Box	5×5	27.819
Gaussian	Box	7×7	26
Gaussian	Gaussian	3×3	31.082
Gaussian	Gaussian	5×5	30.434
Gaussian	Gaussian	7×7	28.871
Gaussian	Median	3×3	29.913
Gaussian	Median	5×5	28.434
Gaussian	Median	7×7	26.656
Salt-and-pepper	Box	3×3	25.821
Salt-and-pepper	Box	5×5	26.1
Salt-and-pepper	Box	7×7	25.154
Salt-and-pepper	Gaussian	3×3	25.373
Salt-and-pepper	Gaussian	5×5	26.641
Salt-and-pepper	Gaussian	7×7	26.752
Salt-and-pepper	Median	3×3	33.948
Salt-and-pepper	Median	5×5	29.386
Salt-and-pepper	Median	7×7	27.002

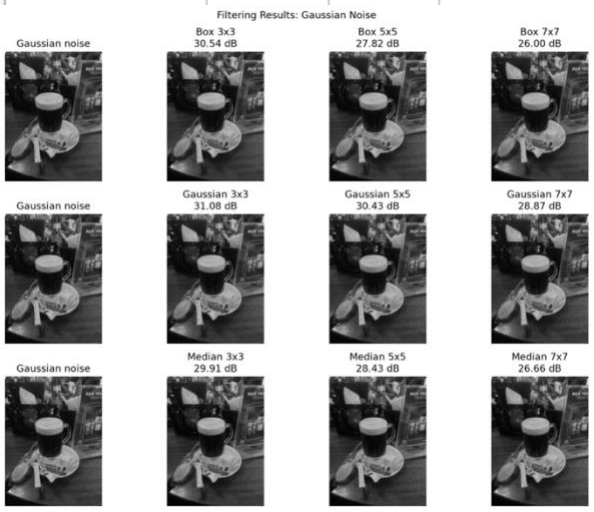

Figure 4: Box, Gaussian and median filtering results for Gaussian noise.

For Gaussian noise, the 3×3 Gaussian filter achieved the highest PSNR of 31.082 dB because its weighted averaging reduced intensity variations while preserving detail. For salt-and-pepper noise, the 3×3 median filter performed best at 33.948 dB because it removed extreme black and white pixels without averaging them into neighboring pixels. Larger kernels reduced PSNR because they caused greater loss of image detail.

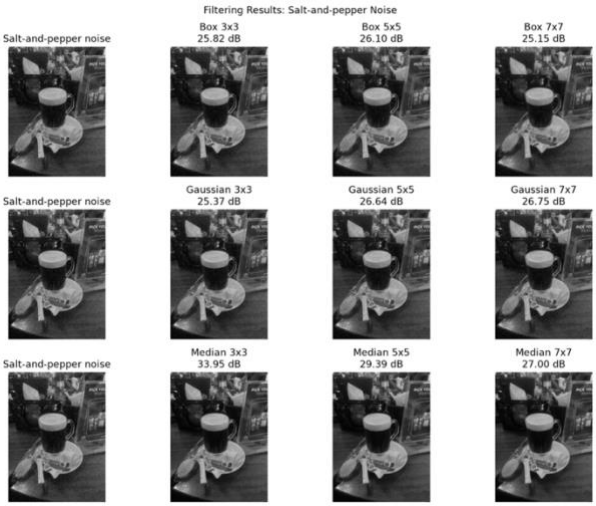

Figure 5: Box, Gaussian and median filtering results for salt-and-pepper noise.

TASK 3: SHARPENING WITH UNSHARP MASKING

Unsharp masking was applied by subtracting a 5×5 Gaussian-blurred image from the original cake image to obtain a detail mask. The mask was multiplied by amounts of 0.5, 1.0 and 2.0 and added to the original. Pixel values were clipped to 0–255 to prevent overflow.

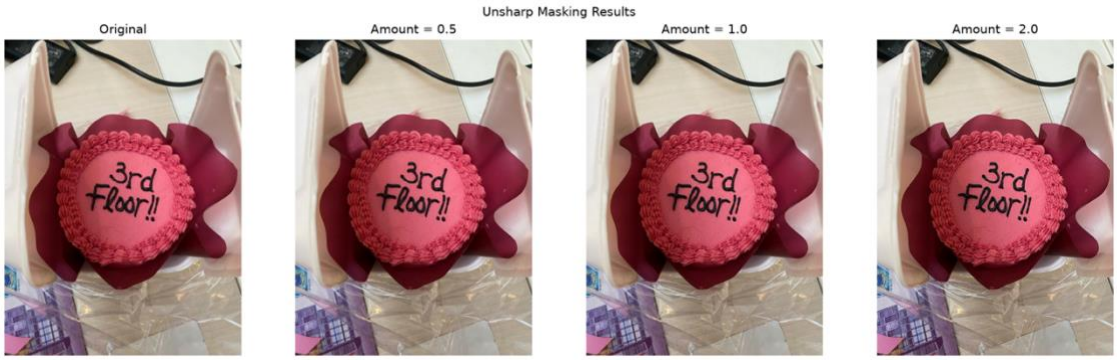

Figure 6: Original image and unsharp-masking results for amounts 0.5, 1.0 and 2.0.

An amount of 0.5 produced mild sharpening, while 1.0 made the lettering and boundaries clearer. An amount of 2.0 created strong outlines and visible halo artefacts. Therefore, increasing the sharpening amount enhances detail but can also exaggerate noise and produce unnatural edges.

TASK 4: EDGE DETECTION THREE WAYS

Part A: Sobel Edge Detection

A custom NumPy convolution applied the horizontal and vertical 3×3 Sobel kernels to the grayscale image. The gradient magnitude was calculated as:

$G = \sqrt{(Gx^2 + Gy^2)}$

The result was compared with cv2.Sobel using identical border handling. The maximum absolute difference was 0.000000, confirming that the custom and OpenCV implementations produced matching results.

Figure 7: Grayscale image, custom NumPy Sobel result and OpenCV Sobel result.

Part B: Canny Edge Detection

Canny detection was tested using threshold pairs (50, 100), (100, 200) and (150, 250). Through hysteresis, pixels above the high threshold are strong edges, while pixels between the thresholds are retained only when connected to strong edges. Lower thresholds detected more detail and unwanted responses, whereas higher thresholds retained only stronger boundaries.

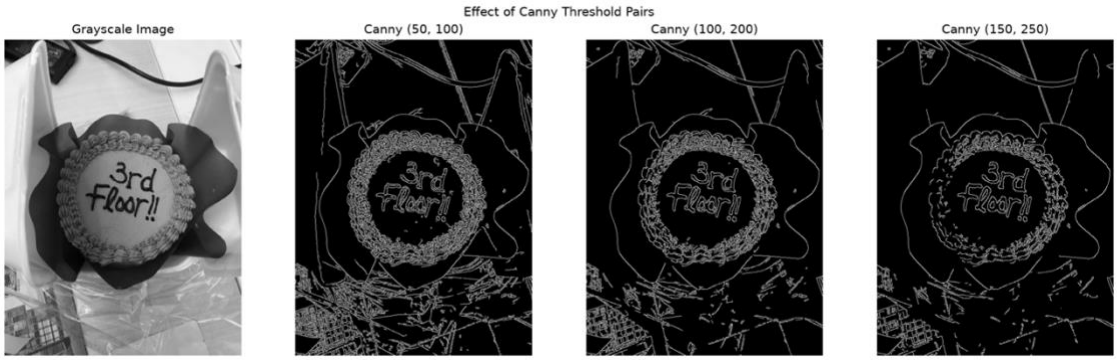

Figure 8: Canny edge maps generated using three threshold pairs.

Part C: Laplacian of Gaussian

A 5×5 Gaussian filter was applied before the Laplacian to reduce noise. Sobel measures directional first-order intensity changes, while LoG uses a second derivative, responds in all directions and can produce finer or double-edge responses.

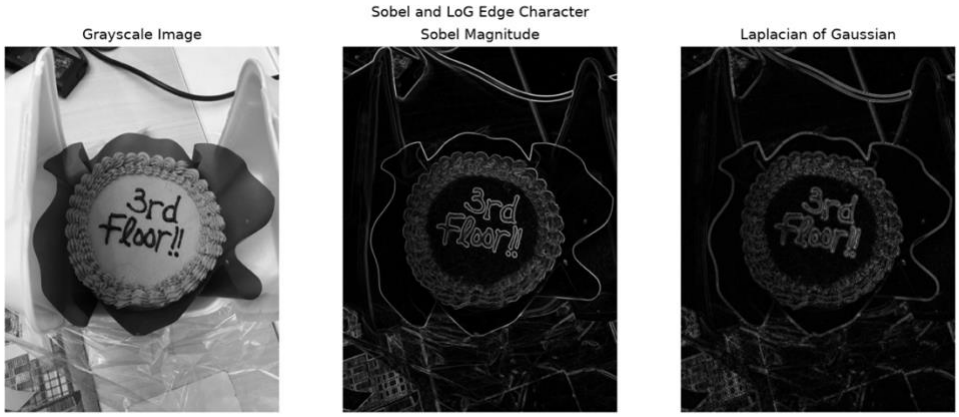

Figure 9: Comparison of Sobel gradient magnitude and Laplacian of Gaussian edges.

TASK 5: MORPHOLOGY – FROM MASK TO OBJECT COUNT

The lotion image was converted to LAB and thresholded using b > 140 for the beige bottle and L > 185 for the white bottle. A 9×9 elliptical structuring element was selected to remove small regions from the textured background while preserving the bottles' rounded shapes.

Figure 10: Original image, initial threshold and the erosion, dilation, opening and closing stages.

Erosion and opening removed small foreground regions, while dilation and closing restored object boundaries and filled small gaps. Connected-component analysis with 8-connectivity was then applied to the cleaned mask, with small components rejected using area and height conditions.

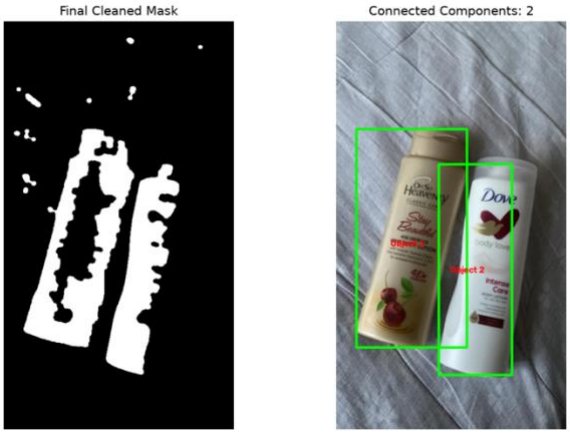

Figure 11: Final cleaned mask and labelled connected components.

The system correctly counted two objects. However, the fixed LAB thresholds are image-specific and may not perform reliably with different objects, backgrounds or lighting.

TASK 6: COMMAND-LINE PIPELINE AND REFLECTION

A command-line tool named process.py was developed to perform color picking, denoising, sharpening, edge detection and object counting. It accepts the input path, operation, optional kernel size and output path. All operations ran successfully, with reproduction commands documented in README.md.

Figure 12: Example edge-detection output produced by the command-line image-processing tool.

Reflection

The HSV color picker selected the pink cake but also included parts of the burgundy wrapper because the two regions had overlapping hue values. The fixed threshold processes pixels according to color and cannot recognize that the cake and wrapper are different objects.

The result is also affected by lighting because shadows, reflections and camera processing change recorded pixel values. A threshold selected for this image may therefore fail under different lighting.

This relates to computer vision being ill-posed: different objects can produce similar image measurements, while the same object can appear different under changing conditions. Color alone cannot always determine object identity.

The pipeline could be improved by combining color with shape and spatial information. Morphology and connected components could remove unrelated regions, while adaptive thresholds or color calibration could improve performance under varying lighting. A trained detector could generalize better but would require labelled data and more computational resources.

References.

[1] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Cham, Switzerland: Springer, 2022.
[2] OpenCV, “OpenCV documentation,” 2026. [Online]. Available: https://docs.opencv.org/. [Accessed: Aug. 31, 2026].
[3] NumPy Developers, “NumPy documentation,” 2026. [Online]. Available: https://numpy.org/doc/. [Accessed: Aug. 31, 2026].
[4] OpenAI, “ChatGPT,” 2026. [Online]. Available: https://chatgpt.com/. [Accessed: Aug. 31, 2026].